

Practice with Flask Part 1



Skills
Network

Estimated time needed: 20 minutes

Welcome to the first lab of the Capstone course. You will practice working with Flask in this lab. You should know all the concepts you need for this lab from the previous set of videos. Feel free to pause the lab and review the module if you are unclear on how to perform a task or need more information.

Learning Objectives

After completing this lab, you will be able to:

- Create and run a Flask server in development mode
 - Return JSON from an endpoint
 - Understand the request object
-

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. Theia is an open source IDE (Integrated Development Environment) that can run on desktop or the cloud. To complete this lab, you will be using the Cloud IDE based on Theia and MongoDB running in a Docker container.

Important Notice about this lab environment

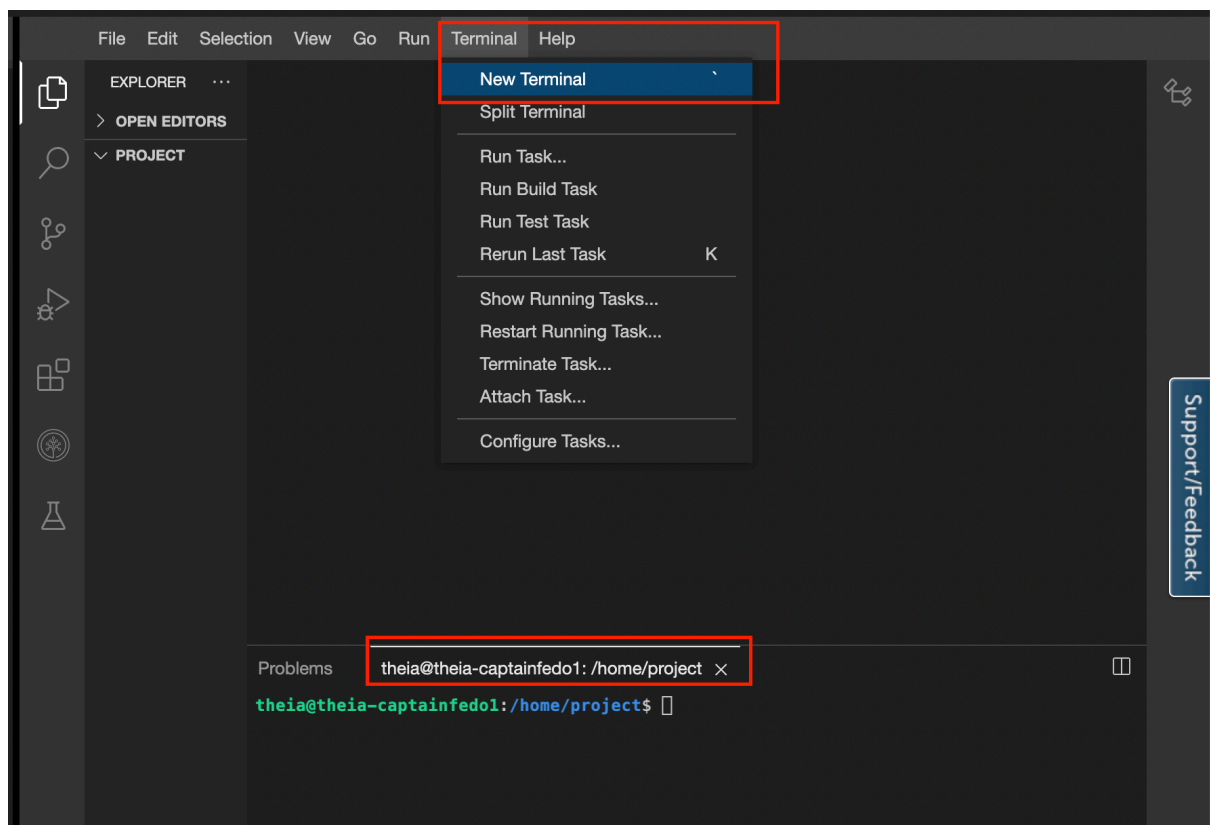
Please be aware that sessions do not persist for this lab environment. Every time you connect to this lab, a new environment is created for you. Any data you save in earlier sessions will be lost. Plan to complete these labs in a single session, to avoid losing your data.

Set Up the Lab Environment

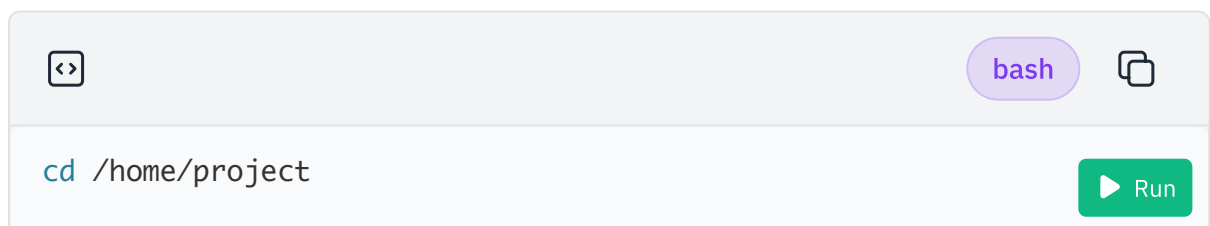
There are some prerequisite preparations required before you start the lab.

Open a Terminal

Open a terminal window using the menu in the editor: **Terminal > New Terminal**.



In the terminal, if you are not in the `/home/project` folder, change to your project folder now.



Create the lab directory

You can now create a directory for your server file

bash

```
mkdir lab
```

Run

Change into the **lab** directory:

bash

```
cd lab
```

Run

Check Python version and install Flask

Use the **python3 --version** command to check the version of python3 in the lab environment. You should see an output as follows:

plaintext

```
theia@theiadocker-captainfedo1:/home/project/lab$ python3 --version  
Python 3.10.12
```

Next, install Flask using the following command:

bash

```
pip3 install flask
```

Run

If Flask is present on the system, you will see the following message:

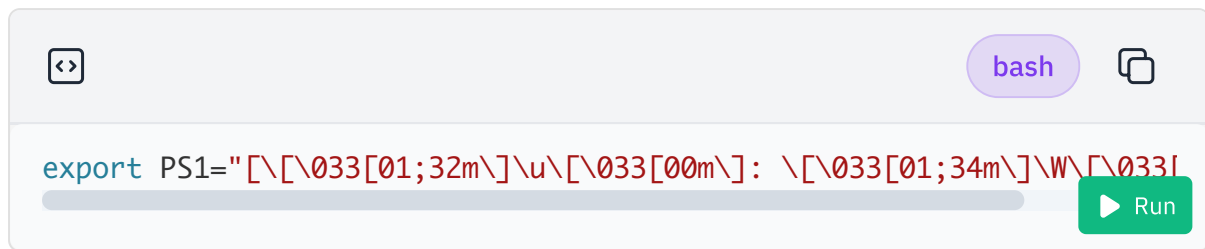
plaintext

```
Requirement already satisfied: flask in /home/theia/.local/lib/python3.10/site-packages (2.0.1)  
Requirement already satisfied:  
...
```

You are now ready to start the lab.

Optional

If working in the terminal becomes difficult because the command prompt is long, you can shorten the prompt using the following command:



```
export PS1="\[\033[01;32m\]\u\[\033[00m\]: \[\033[01;34m\]\W\[\033[00m\]"
```

The image shows a terminal window with a light gray header bar. On the left is a code icon, and on the right is a 'bash' button and a copy icon. The terminal content shows the command `export PS1="\[\033[01;32m\]\u\[\033[00m\]: \[\033[01;34m\]\W\[\033[00m\]"` being entered. A green 'Run' button is at the bottom right of the terminal area.

Step 1: Create the Hello World server

Your Tasks

1. Create server.py file.

First, create an empty file called `server.py` in the terminal or use the file editor menu.

▶ Click here for a hint.

Open `server.py` in the editor

Open server.py in IDE

If a new tab called `Python – Get Started` displays after opening this file, you can close it to return to the python file.

2. Import Flask module.

Next, import the Flask module in this file so you can start coding the server.

▶ Click here for a hint.

3. Create the Flask app

After importing the Flask module, create your Flask application by initializing the Flask class.

▶ Click here for a hint.

4. Create the main route and define its method. Import Flask, then use the app you created in the previous task to create your first route for the root URL and

► Click here for a hint.

5. Return the "Hello World" message to the client.

Return the string "Hello World" to the client.

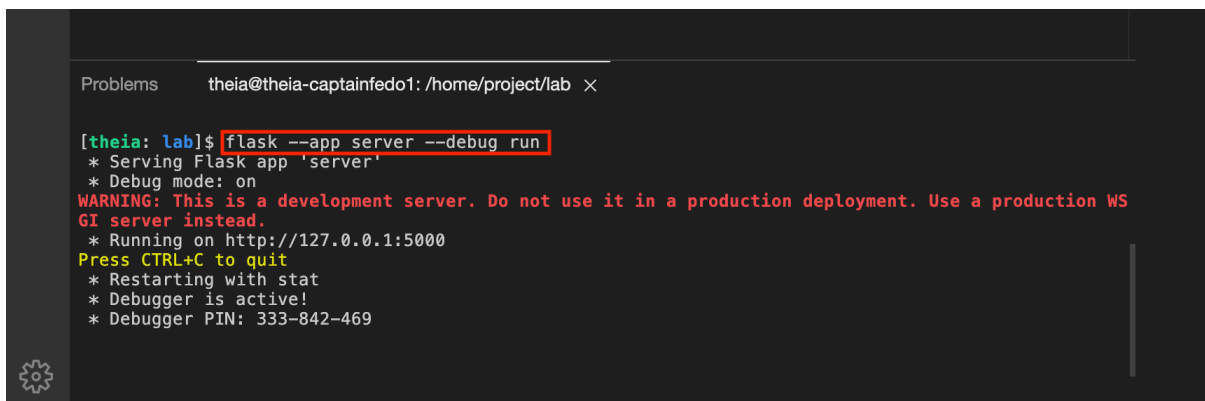
► Click here for a hint.

You are all set to run the server. Use the following command to run the server from the terminal:

 bash 

```
flask --app server --debug run
```





```
theia@theia-captainfedo1: /home/project/lab x
[theia: lab]$ flask --app server --debug run
* Serving Flask app 'server'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 333-842-469
```

You should now be able to use the CURL command on `localhost:5000/`. Note that the terminal is already running the server, you can use the **Split Terminal** button to split the terminal and run the following command in the second tab.

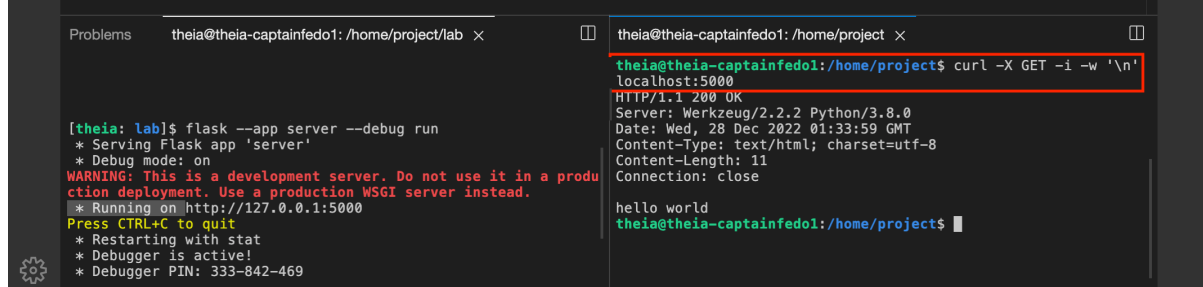
Note: Kindly verify the presence of the `Server.py` file in the `/home/project/lab` directory to prevent encountering a connection refusal error.

 bash 

```
curl -X GET -i -w '\n' localhost:5000
```



The `-X` argument specifies the `GET` command, and the `-i` argument displays the header from the response.



The screenshot shows a VS Code interface with two terminal windows. The left window, titled 'theia@theia-captainfedo1: /home/project/lab', shows the command `flask --app server --debug run` being executed. The output indicates that the Flask app 'server' is running in debug mode on `http://127.0.0.1:5000`. A warning message states: 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.' The right window, titled 'theia@theia-captainfedo1: /home/project', shows the command `curl -X GET -i -w '\n' localhost:5000` being executed. The output of the curl command is: `HTTP/1.1 200 OK`, `Server: Werkzeug/2.2.2 Python/3.8.0`, `Date: Wed, 28 Dec 2022 01:33:59 GMT`, `Content-Type: text/html; charset=utf-8`, `Content-Length: 11`, and `Connection: close`. The final output line is `hello world`.

You should see **Hello World** returned as the output of the CURL command. Note the return status of **HTTP 200 OK** and the **Content-type** of **text/html**. You are asked to return a custom status with JSON instead of plain text in the next part of this lab.

Solution

Double-check that your work matches the solution below.

► [Click here for the answer.](#)

Here's the updated instruction with **jsonify** removed from the hint's import:

Step 2: Return JSON

Your Task

Congratulations on creating your first route handler in the Flask server. You can return a number of different content types from the `@app.route()` methods. For the purpose of this project, let's return the following JSON instead of the **Hello World** string.

 json 

```
"message": "Hello World"
```

Recall from the videos that there are two ways to return a JSON object from the method:

1. Return a Python dictionary
2. Use the `jsonify()` method on a string

You are being asked to use the first method in this lab.

Hint

You can edit the existing **index** method to return the desired JSON message.

▶ Click here for a hint.

Solution

Double-check that your work matches the following solution.

▶ Click here for the answer.

If you have the server running, you are good to go. If not, you can run the server with the following command again:

bash

```
flask --app server --debug run
```

Run

You should now be able to use the CURL command with `localhost:5000/`. Note that the terminal is running the server, you can use the **Split Terminal** button to split the terminal and run the following command in the second tab.

bash

```
curl -X GET -i -w '\n' localhost:5000
```

Run

```
theia@theia-captainfedo1:/home/project$ curl -X GET -i -w '\n' localhost:5000
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.8.0
Date: Wed, 28 Dec 2022 18:12:32 GMT
Content-Type: application/json
Content-Length: 31
Connection: close

{
  "message": "Hello World"
}
```

You should see `{"message": "Hello World"}` JSON returned as the output of the CURL command. Note the return status of `HTTP 200 OK` and the `Content-type` of `application/json` this time.

Author(s)

CF

© IBM Corporation 2023. All rights reserved.

Step 1: Create the Hello World

server

Your Tasks

1. Create server.py file.

First, create an empty file called `server.py` in the terminal or use the file editor menu.

▼ Click here for a hint.

The following command will create the empty file in the right directory.



bash



```
touch /home/project/lab/server.py
```

▶ Run

Open `server.py` in the editor

Open server.py in IDE

If a new tab called `Python – Get Started` displays after opening this file, you can close it to return to the python file.

2. Import Flask module.

Next, import the Flask module in this file so you can start coding the server.

▶ Click here for a hint.

3. Create the Flask app

After importing the Flask module, create your Flask application by initializing the Flask class.

▼ Click here for a hint.

Initiate a new application from the Flask class.



python



```
from flask import <module_name>
app = <module_name>(__name__)
```

4. Create the main route and define its method. Import Flask, then use the app you created in the previous task to create your first route for the root URL and define its handler method.

► Click here for a hint.

5. Return the "Hello World" message to the client.

Return the string "Hello World" to the client.

► Click here for a hint.

You are all set to run the server. Use the following command to run the server from the terminal:

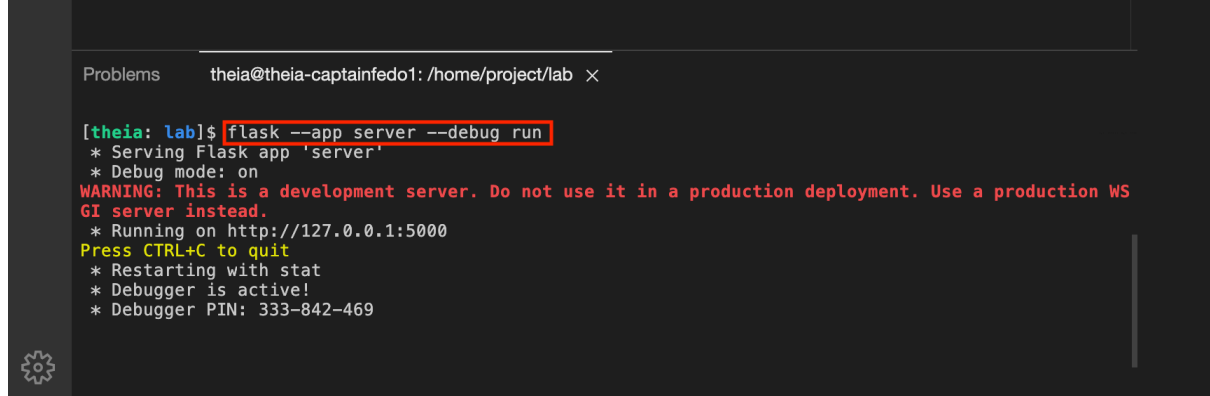


bash



```
flask --app server --debug run
```

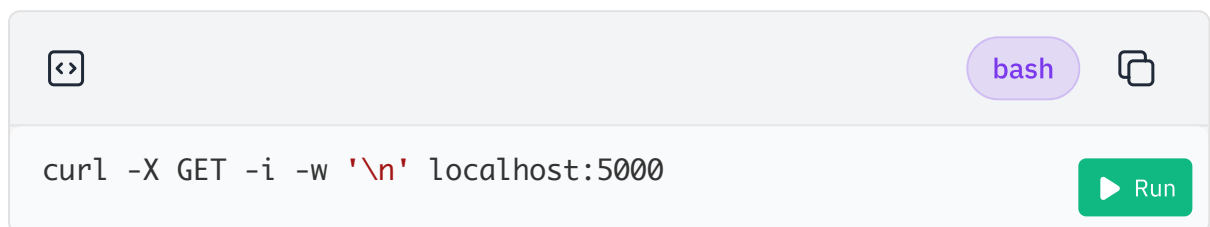
▶ Run

A terminal window with a dark background. The prompt is 'theia@theia-captainfedo1: /home/project/lab'. The command 'flask --app server --debug run' is entered and highlighted with a red box. The output shows the Flask server starting in debug mode, listening on http://127.0.0.1:5000, and displaying a warning about production deployment. The prompt returns to the user.

```
theia@theia-captainfedo1: /home/project/lab ×  
[theia: lab]$ flask --app server --debug run  
* Serving Flask app 'server'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5000  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!  
* Debugger PIN: 333-842-469
```

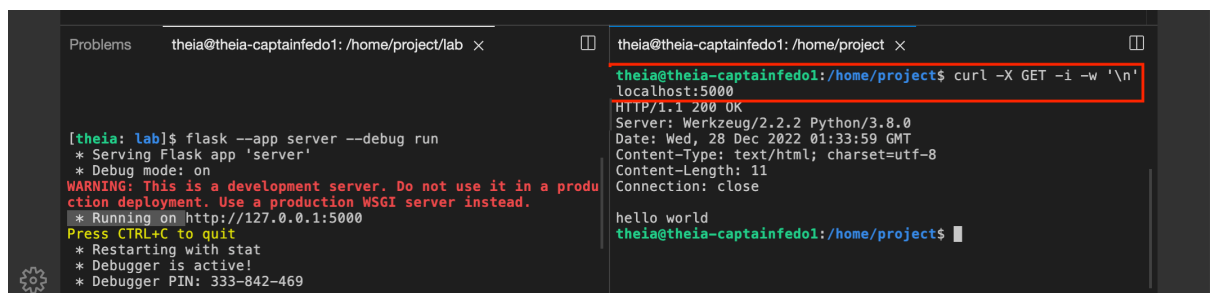
You should now be able to use the CURL command on `localhost:5000/`. Note that the terminal is already running the server, you can use the **Split Terminal** button to split the terminal and run the following command in the second tab.

Note: Kindly verify the presence of the `Server.py` file in the `/home/project/lab` directory to prevent encountering a connection refusal error.

A light gray code editor snippet. It has a 'bash' button and a copy icon on the right. The command 'curl -X GET -i -w '\n' localhost:5000' is entered. A green 'Run' button is on the right.

```
bash  
curl -X GET -i -w '\n' localhost:5000  
Run
```

The `-X` argument specifies the `GET` command, and the `-i` argument displays the header from the response.

A split terminal window. The left pane shows the Flask server running. The right pane shows the output of the curl command, which returns 'HTTP/1.1 200 OK' and 'hello world'. The curl command and its output are highlighted with a red box.

```
theia@theia-captainfedo1: /home/project/lab ×  
[theia: lab]$ flask --app server --debug run  
* Serving Flask app 'server'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5000  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!  
* Debugger PIN: 333-842-469  
  
theia@theia-captainfedo1: /home/project ×  
theia@theia-captainfedo1: /home/project$ curl -X GET -i -w '\n' localhost:5000  
HTTP/1.1 200 OK  
Server: Werkzeug/2.2.2 Python/3.8.0  
Date: Wed, 28 Dec 2022 01:33:59 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 11  
Connection: close  
  
hello world  
theia@theia-captainfedo1: /home/project$
```

You should see `Hello World` returned as the output of the CURL command. Note the return status of `HTTP 200 OK` and the `Content-type` of `text/html`. You are asked to return a custom status with JSON instead of plain text in the next part of this lab.

Solution

Double-check that your work matches the solution below.

▼ Click here for the answer.



python



```
# Import the Flask class from the flask module
from flask import Flask

# Create an instance of the Flask class, passing in the name of
app = Flask(__name__)

# Define a route for the root URL ("/")
@app.route("/")
def hello_world():
    # Function that handles requests to the root URL
    return "Hello, World!"
```

Here's the updated instruction with `jsonify` removed from the hint's import:

← Set Up the Lab Environm...

Step 2: Return JSON →